

NL2SQL Agent study handbook

NL2SQL Agent study handbook

How to use this handbook:

1. Read modules in order. Each module builds on the previous one.
2. Keep the code open while reading and follow the file and function references.
3. After Module 5, use the checklist to review what you have learned.

Module 1: Foundations and architecture

1.1 What this project does

`n12sql-agent` is a local-first natural-language-to-SQL system. A user asks a plain-English question (CLI or Streamlit UI), the agent generates SQL, validates it for safety, runs it on read-only SQLite or operator-configured PostgreSQL, and returns a natural-language answer plus SQL/result context.

Primary use cases in this repo:

1. Local analytics assistant over SQLite (`company.db` by default) or a restricted PostgreSQL schema.
2. Safe text-to-SQL experimentation with policy controls (joins, subqueries, aggregates, CTE).
3. Multi-provider runtime where Ollama is default and Hugging Face, OpenAI, Anthropic, Gemini, xAI, and Agnes AI are optional.
4. Approval-first analysis of session-scoped uploaded SQLite databases.
5. Result-based model evaluation across accuracy, safety, latency, retries, and token usage.

1.2 Core paradigms and patterns used

Definitions first, then where they appear in this codebase:

1. Modular architecture: Definition: split the system into small modules with one clear job each. In this repo: `config, db, security, llm, prompts, agent, ui, utils` under `src/n12sql_agent/`.
2. Factory pattern: Definition: one function returns different implementations behind a shared interface. In this repo: `build_chat_model()` returns provider-specific chat models in `src/n12sql_agent/llm/factory.py`.
3. Stateful workflow (graph/state-machine orchestration): Definition: execution is modeled as nodes and conditional transitions over shared state. In this repo: `NL2SQLAgent.get_workflow()` in `src/n12sql_agent/agent/workflow.py` using LangGraph.
4. Policy-based security (allow-list): Definition: explicitly allow known-safe behavior instead of only blocking known-bad patterns. In this repo: `SQLPolicy + validate_sql()` in `src/n12sql_agent/security/sql_validator.py`.
5. Typed data contracts: Definition: key data objects have explicit types and field names. In this repo: `AgentState` (`agent/state.py`) and `QueryResult` (`db/database.py`).
6. Settings singleton with environment binding: Definition: one cached settings object is reused across the app and can be reset in tests. In this repo: `Settings, get_settings(), reset_settings_cache()` in `config/settings.py`.
7. Defense in depth: Definition: multiple independent controls prevent one failure from becoming data loss. In this repo: AST validation, table authorization, preflight compilation, read-only SQLite, progress limits, and redacted audits.

1.3 Architecture description

Components and interactions:

1. Entrypoints:
 - CLI: `src/nl2sql_agent/cli.py` (`main()`, subcommands `ask/config/serve/eval`)
 - UI: `src/nl2sql_agent/ui/streamlit_app.py` (`main()`)
2. Model construction:
 - `build_chat_model()` selects provider and returns a `BaseChatModel`.
3. Core agent workflow:
 - `NL2SQLAgent` builds a `LangGraph` with nodes: `fetch_schema` -> `writer` -> `guardian` -> `executor` -> `summarizer`
4. Data access:
 - `Database.ensure_schema()` creates/seeds tables.
 - `Database.execute()` runs SQL on a hardened read-only connection and returns `QueryResult`.
5. Security gate:
 - `prepare_sql()` blocks non-SELECT, multi-statement, dangerous functions, disallowed tables, and policy violations, then enforces the row limit.
6. Prompting:
 - SQL-writing and summarization prompts in `prompts/templates.py`.

1.4 Main flow diagram (ASCII)

```
User (CLI ask / Streamlit chat / eval corpus)
  |
  v
build_chat_model(settings, provider?, model?)
  |
  v
NL2SQLAgent(llm, settings)
  |
  v
LangGraph workflow:

START
-> fetch_schema      (db.get_schema_text)
-> writer            (LLM creates sql_query)
-> guardian          (prepare_sql + EXPLAIN QUERY PLAN)
    | safe                    | preflight error and attempts remain
    v                        v
  executor              writer
(db.execute)          (LLM SQL rewrite)
    | ok
    | error and retry_count < max_retries
    +-----> writer (retry)

summarizer -> END

Streamlit stops after guardian, shows an editable SQL preview, then calls
execute_prepared(), which validates again before executor.
```

Schema fetching keeps one read-only connection open, scores every table and column identifier once, and loads foreign-key metadata only for the tables that will be sent to the writer.

Module 2: Repository map

Files a new contributor should learn first (in practical reading order):

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
Launch NL2SQL Agent.cmd	Windows double-click startup	checks uv, then runs nl2sql-agent serve	locked environment, loopback port 8512, visible logs
pyproject.toml	Packaging, dependencies, scripts, lint/type/test config	[project.scripts] nl2sql-agent	requires-python, direct dependencies, transitive constraints, Ruff/ty/Pytest settings
src/nl2sql_agent/cli.py	CLI entrypoint for ask/config/serve/eval	main, cmd_ask, cmd_eval, cmd_config, cmd_serve	Provider/model overrides, eval threshold, report and cost options
src/nl2sql_agent/ui/streamlit_app.py	Approval-first multipage Streamlit orchestration	main, _resolve_database, _build_agent	Chat context, temporary upload workspace, pending SQL, active database context
src/nl2sql_agent/ui/components.py	UI widgets and result rendering	render_sidebar, render_chat_history, render_run_result	Database source, provider/model, results, traces, plans, metrics, CSV download
src/nl2sql_agent/ui/pages.py	Costs, Sessions, Insights, and Pricing views	costs_page, sessions_page, insights_page, pricing_page	Dashboards, saved-session management, rule editing, budgets
src/nl2sql_agent/persistence.py	Versioned local state database	StateStore	Verbatim messages, pending SQL, usage, pricing and metrics; structured rows excluded, answer values retained
src/nl2sql_agent/ui/database_upload.py	Untrusted upload validation/storage	validate_sqlite_upload, save_sqlite_upload	Extension, size, SQLite header, content digest
src/nl2sql_agent/agent/workflow.py	Full and two-phase workflow routing	NL2SQLAgent, prepare, stream_prepare, execute_prepared, run, stream	Run ID, allowlist, SQL safety, retries, traces, token usage
src/nl2sql_agent/agent/state.py	Typed workflow contract	AgentState	Fields for schema, SQL, retries, raw rows, columns, row_count
src/nl2sql_agent/security/sql_validator.py	SQL AST validation and executable preparation	SQLPolicy, validate_sql, prepare_sql, referenced_tables	Feature toggles, structural limits, table allowlist, enforced LIMIT
src/nl2sql_agent/db/database.py	Read-only SQLite schema/query layer	Database, QueryResult, list_tables, get_schema_text, preflight, execute	Timeout, VM steps, row limit, normalized plans/metrics, schema ranking
src/nl2sql_agent/db/postgres.py	Strict read-only PostgreSQL layer	PostgresDatabase	Restricted role verification, one-schema introspection, timeouts, JSON plans
src/nl2sql_agent/evaluation/runner.py	Result and safety evaluation	EvalCase, EvaluationRunner, EvaluationReport	Reference result comparison, threshold metrics, integrity digest
src/nl2sql_agent/utils/audit.py	Privacy-preserving JSONL audit	AuditLogger, redact_sql, hash_text	Hashed questions, literal-redacted SQL
src/nl2sql_agent/db/seed.py	Demo seed data	SEED_DEPARTMENTS, SEED_EMPLOYEES	Department/employee tuples loaded via INSERT OR IGNORE

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
src/nl2sql_agent/llm/factory.py	Provider-specific model building and model listing	build_chat_model, list_models, fallback_models, _build_*, _list_ollama, LLMProviderError	Seven providers; documented hosted reasoning modes; Ollama-only live discovery
src/nl2sql_agent/llm/pricing.py	Per-call hosted-model pricing	PricingRule, UsageRecord, calculate_cost	Effective dates, cache/batch/fast rates, long-context thresholds, exact decimal arithmetic
src/nl2sql_agent/config/settings.py	Central runtime config and model policy	Settings, default_model_for, supported_models_for, validate_model_for, env_var_for	env_prefix="NL2SQL_", provider/model allow-lists, credentials, DB/SQL/log fields
src/nl2sql_agent/prompts/templates.py	Prompt templates and formatting helpers	SQL_WRITER_SYSTEM, SQL_WRITER_USER, SUMMARIZER_SYSTEM, SUMMARIZER_USER, error_section, format_data	Prompt placeholders {schema}, {question}, {error_section}, {sql}, {data}, {error}
src/nl2sql_agent/utils/text.py	SQL/text normalization utilities	strip_sql_fences, truncate	Regex constants _SQL_FENCE_RE, _LEADING_SQL_TOKEN_RE
src/nl2sql_agent/utils/logging.py	Unified Loguru logging setup	configure_logging, get_logger	level, json, stdlib interception for httpx/httpcore/...
API_REFERENCE.md	Exported Python and CLI reference	Public constructors, methods, errors, and examples	Generated from the current package exports and signatures
diagrams/	Interactive system views	Architecture, workflow, sequence, data flow, lifecycle	Standalone HTML with editable JSON specifications

Module 3: Core execution flows

3.1 Core data contracts

AgentState shape (src/nl2sql_agent/agent/state.py)

```
# TypedDict, total=False
{
    "run_id": str,
    "question": str,
    "schema": str,
    "allowed_tables": list[str],
    "sql_query": str,
    "sql_safe": bool,
    "sql_unsafe_reason": str,
    "result": str,
    "raw_rows": list[tuple[object, ...]],
    "columns": list[str],
    "row_count": int,
    "csv_data": str,
    "truncated": bool,
    "error": str,
    "retry_count": int,
    "max_retries": int,
    "trace": list[dict[str, object]],
    "token_usage": dict[str, int],
}
```

```
"final_answer": str,
}
```

AgentState uses total=False, so nodes can return partial updates.

QueryResult shape (src/nl2sql_agent/db/database.py)

Field outline (not a constructor call):

```
QueryResult(
    columns: tuple[str, ...],
    rows: tuple[tuple[object, ...], ...],
    row_count: int,
    truncated: bool = False,
    metrics: QueryMetrics = field(default_factory=QueryMetrics)
)
```

Sidebar config payload (src/nl2sql_agent/ui/components.py)

```
{
    "data_source": str,           # "Demo" | "Upload" | "PostgreSQL"
    "provider": Provider,         # ollama | huggingface | openai | anthropic | gemini | xai | agnes
    "model": str | None,
    "api_key": str | None,
}
```

Initial workflow input shape (run/stream in src/nl2sql_agent/agent/workflow.py)

The following is a partial outline of _initial_state(), shared by run and preparation APIs. It also initializes outcome, execution, clarification, usage records, warnings and provider/model fields:

```
{
    "run_id": str(uuid4()),
    "question": question,
    "retry_count": 0,
    "max_retries": int(max_retries if max_retries is not None else self.settings.max_retries),
    "error": "",
    "trace": [],
    "token_usage": {},
    "allowed_tables": sorted(self.allowed_tables),
}
```

3.2 Flow A: CLI ask request

Entrypoint chain:

1. main() parses args via build_parser().
2. cmd_ask(args) calls _build_agent(...).
3. _build_agent:
 - loads settings = get_settings()
 - applies CLI overrides (provider/model/api_key)
 - builds llm = build_chat_model(settings)
 - returns NL2SQLAgent(llm, settings=settings)
4. cmd_ask calls agent.run(args.question), passing clarification replies when supplied.
5. Prints result["final_answer"]; if --show-sql, also prints result["sql_query"].

Simplified fragment (without the optional clarification branch):

```

result = agent.run(args.question)
print(result.get("final_answer", ""))
if args.show_sql:
    print("\n--- SQL ---\n", result.get("sql_query", ""), sep="")

```

Input shape: - args.question: str - optional overrides: args.provider, args.model, args.api_key

Output shape: - terminal text; underlying result dict includes final_answer, sql_query, error, columns, raw_rows, row_count.

3.3 Flow B: Streamlit chat request

Entrypoint chain:

1. streamlit_app.main() configures logging, local state, and five native pages.
2. render_sidebar() selects Demo/Upload/PostgreSQL and the model provider.
3. Uploads are validated, stored under a content digest, and opened read-only.
4. The user selects allowed tables and whether uploaded sample rows may enter prompts.
5. agent.stream_prepare(user_query) emits schema, writer, and guardian updates.
6. Safe SQL is stored as pending_query and shown in an editable text area.
7. Run calls agent.execute_prepared(...), which validates the edited SQL again.
8. Structured rows and CSV remain in memory. Saved answer text can still contain result values. Pending approval retains questions, clarifications and SQL.
9. _apply_cost() prices usage with the effective local rule; render_run_result() displays the resulting estimate and runtime warnings.

UI result-rendering excerpt:

```

for node_name, update in agent.stream_prepare(user_query):
    final_state.update(update)

result = agent.execute_prepared(final_state, sql_query=edited_sql)

```

Input shape: - user_query: str - sidebar config from render_sidebar()

Output shape: - Editable SQL preview, answer, dataframe, CSV, stage timings, token counts, normalized plan/metrics, warnings, and an estimated hosted-model cost.

Each model call records provider-reported input/output/cache usage. The effective local rule applies its actual request mode and long-context threshold, then freezes a price snapshot with the approved run. Estimates remain local guidance rather than provider invoices.

3.4 Flow C: Agent internals (LangGraph node-by-node)

NL2SQLAgent.get_workflow() defines this graph:

1. fetch_schema(state):
 - ranks allowed tables against the question and includes FK neighbors
 - optionally includes bounded sample rows
 - returns schema, allowlist, and a timing trace
2. write_sql(state):
 - reads: question, schema, previous error/sql_unsafe_reason
 - builds SQL_WRITER_USER prompt
 - LLM call with system+user messages
 - parses a structured sql, clarify, or unanswerable decision

- validates JSON fields; SQL-only responses remain a compatibility path
 - ends without execution on clarification, unanswerable, or provider failure
 - returns:
 - `sql_query`
 - incremented `retry_count`
 - clears stale error, `sql_unsafe_reason`, result
3. `check_security(state)`:
- runs `prepare_sql(sql, policy, allowed_tables)`
 - canonicalizes and enforces the executable LIMIT
 - runs `Database.preflight()` using `EXPLAIN QUERY PLAN`
 - records a redacted prepared or blocked audit event
4. `route_after_security(state)`:
- if safe -> "executor"
 - if generation/preflight failed and retry budget remains -> "writer"
 - policy blocks, provider failures, and clarification decisions do not retry SQL
 - otherwise -> "summarizer"
5. `execute_sql(state)`:
- calls `self.db.execute(sql)`
 - on success returns markdown, rows, columns, count, CSV, truncation state, and timing
 - on `sqlite3.Error` returns error string and empty data containers
6. `route_after_execute(state)`:
- if error and `retry_count < max_retries` -> "writer"
 - otherwise -> "summarizer"
7. `summarize_result(state)`:
- renders successful results directly from database cells
 - preserves `result` as the raw Markdown table and sets `final_answer`
 - reports SQL-limit, fetch-truncation, and preview-sampling limits separately
 - renders clarification, unanswerable, and error messages without another model call

`prepare()/stream_prepare()` use a smaller graph that ends after guardian. `execute_prepared()` re-enters guardian before executor. `run()` and `stream()` continue to use the end-to-end graph for automation compatibility.

3.5 Flow D: Database initialization + seed behavior

Where it happens:

1. For a managed demo in `NL2SQLAgent.__init__`:
 - `self.db = Database(...)`
 - `self.db.ensure_schema(seed=self.settings.db_seed)`

Injected databases are never initialized or seeded. This is the path used for uploads.

2. `Database.ensure_schema(seed=True)`:
 - creates parent directories
 - executes DDL for departments and employees
 - inserts `SEED_DEPARTMENTS` and `SEED_EMPLOYEES` using `INSERT OR IGNORE`
 - guarded by `_init_lock` and `_initialized` flag for idempotency

Tables and seed columns:

1. `departments(dept_id, dept_name, location)`
2. `employees(emp_id, name, salary, dept_id)` with FK `employees.dept_id -> departments.dept_id`

3.6 Flow E: Safety gate behavior details

`prepare_sql()` in `security/sql_validator.py` enforces:

1. SQL must parse and be non-empty (`parse_sql`).
2. Exactly one statement.
3. Top-level must be `SELECT` or the explicitly supported `UNION` AST node.
4. Optional policy bans:
 - subqueries
 - joins
 - aggregates
 - CTEs
 - union
5. Dangerous functions are blocked (`load_extension`, `readfile`, `writefile`, `edit`, `shell`, `system`).
6. Additional banned keyword scan blocks destructive operations.
7. `JOIN`, subquery, and CTE counts stay within configured limits.
8. Every referenced physical table belongs to the current allowlist. CTE names are derived sources, but same-name physical tables are still authorized independently through scope traversal.
9. Preparation reuses one parsed AST for validation, table discovery, and `LIMIT` canonicalization; a regression test enforces the single-parse path.
10. The returned SQL contains an enforced result `LIMIT`.

After AST checks, `Database.preflight()` validates identifiers without running the `SELECT`. Execution uses SQLite URI `mode=ro`, `query_only`, disabled extensions, `trusted_schema=OFF`, an elapsed-time deadline, and a VM-step cap.

Output contract: - Success: returns `PreparedSQL(sql=..., tables=...)`. - Failure: raises `SQLValidationError` with user-facing message.

The safety gate enforces the system's trust boundary before query execution. Settings flags map to `SQLPolicy` to configure it.

Module 4: Setup and run guide

This section is static and based on repo files (`README.md`, `pyproject.toml`, `source`).

4.1 Prerequisites

1. Python 3.12.10 (from `pyproject.toml`: `requires-python = ">=3.12.10,<3.13"`).
2. `uv` for environment/dependency management.
3. Optional but default runtime path: local Ollama (`NL2SQL_PROVIDER=ollama`).

4.2 Install on a clean machine

```
git clone https://github.com/pypi-ahmad/natural-language-to-sql-agent.git
cd natural-language-to-sql-agent
uv sync --locked --all-groups
```

4.3 Environment configuration (`.env` or shell `env`)

Core Settings keys (`src/nl2sql_agent/config/settings.py`)

Use these as `NL2SQL_*` variables:

1. `NL2SQL_PROVIDER`

2. NL2SQL_MODEL
3. NL2SQL_OLLAMA_BASE_URL
4. NL2SQL_OLLAMA_KEEP_ALIVE
5. NL2SQL_DB_PATH
6. NL2SQL_DB_SEED
7. NL2SQL_DB_MAX_ROWS
8. NL2SQL_DB_QUERY_TIMEOUT_SECONDS
9. NL2SQL_DB_MAX_VM_STEPS
10. NL2SQL_DB_UPLOAD_MAX_MB
11. NL2SQL_MAX_RETRIES
12. NL2SQL_LLM_TEMPERATURE
13. NL2SQL_LLM_MAX_TOKENS
14. NL2SQL_LLM_REQUEST_TIMEOUT_SECONDS
15. NL2SQL_SQL_ALLOW_SUBQUERIES
16. NL2SQL_SQL_ALLOW_JOINS
17. NL2SQL_SQL_ALLOW_AGGREGATES
18. NL2SQL_SQL_ALLOW_CTE
19. NL2SQL_SQL_MAX_JOINS
20. NL2SQL_SQL_MAX_SUBQUERIES
21. NL2SQL_SQL_MAX_CTES
22. NL2SQL_SCHEMA_MAX_TABLES
23. NL2SQL_AUDIT_ENABLED
24. NL2SQL_AUDIT_PATH
25. NL2SQL_LOG_LEVEL
26. NL2SQL_LOG_JSON

Provider API key env names used by UI helper

env_var_for() in settings.py maps:

1. openai -> OPENAI_API_KEY
2. gemini -> GOOGLE_API_KEY
3. anthropic -> ANTHROPIC_API_KEY
4. huggingface -> HF_TOKEN
5. xai -> XAI_API_KEY
6. agnes -> AGNESAI_API_KEY (legacy aliases remain accepted)

Minimal .env examples:

```
# Local-first default path
NL2SQL_PROVIDER=ollama
NL2SQL_MODEL=phi4-mini:3.8b
NL2SQL_OLLAMA_BASE_URL=http://localhost:11434
NL2SQL_DB_PATH=company.db
NL2SQL_DB_SEED=true
```

```
# Cloud provider example (OpenAI)
NL2SQL_PROVIDER=openai
NL2SQL_MODEL=gpt-5.6-luna
OPENAI_API_KEY=your_key_here
NL2SQL_DB_PATH=company.db
```

Hosted requests use medium reasoning. OpenAI allows gpt-5.6-luna, gpt-5.6-terra, and gpt-6-luna; Anthropic allows claude-sonnet-5; Gemini allows gemini-3.7-flash and gemini-3.5-flash-lite; xAI allows grok-4.6. Hugging Face accepts namespace/model[:routing-policy] through its direct router and defaults to openai/gpt-oss-120b:fastest. Agnes defaults

to agnes-3.0-flash, keeps agnes-2.5-flash, and uses boolean Chat Completions Thinking flag rather than a low/medium/high effort value.

4.4 Typical run commands

Start Streamlit UI

On Windows, double-click `Launch NL2SQL Agent.cmd`. The terminal equivalent is:

```
uv run streamlit run src/nl2sql_agent/ui/streamlit_app.py
```

Ask one question in CLI

```
uv run nl2sql-agent ask "How many employees are in each department?"  
uv run nl2sql-agent ask --show-sql "What is the total salary in Engineering?"
```

Show resolved runtime config

```
uv run nl2sql-agent config
```

Serve command wrapper (invokes Streamlit via CLI)

```
uv run nl2sql-agent serve --host 127.0.0.1 --port 8512
```

Run the result and safety corpus

```
uv run nl2sql-agent eval --min-pass-rate 0.8
```

4.5 Database migration/seeding steps

Demo database creation uses idempotent DDL, not Alembic or Flyway. The separate local state database has a versioned initializer using `SQLite user_version`.

Current behavior:

1. Schema creation is runtime and idempotent via `Database.ensure_schema()`.
2. Seeding is controlled by `db_seed` (`NL2SQL_DB_SEED`, default `True`).
3. Backward-compatible helper exists: `setup_db(path, seed=True)`.
4. Only the managed demo is initialized by the agent. Never point the demo path at a database that should not receive its schema; use an injected backend or the upload/PostgreSQL path for existing data.

Module 5: Study plan and practice exercises

5.1 Ordered study plan (zero to hero)

1. Read `README.md` sections: architecture, workflow, configuration, CLI/UI.
2. Read `src/nl2sql_agent/config/settings.py` to understand runtime knobs.
3. Read `src/nl2sql_agent/db/database.py` + `db/seed.py` for actual data model.
4. Read `src/nl2sql_agent/security/sql_validator.py` for guardrail logic.
5. Read `src/nl2sql_agent/prompts/templates.py` for prompt contracts.

6. Read `src/nl2sql_agent/llm/factory.py` for provider abstraction.
7. Read `src/nl2sql_agent/agent/state.py` then `agent/workflow.py` linearly.
8. Read `src/nl2sql_agent/ui/streamlit_app.py` + `ui/components.py`.
9. Read `src/nl2sql_agent/cli.py`.
10. Confirm the current package state with the verification commands in `README.md`. Use the `tests/` suite for offline regression proof, then the 15-case smoke evaluator for an explicitly authorized live check.

5.2 Practice exercises

1. Trace one full run: Starting from `cmd_ask`, list each function called until `final_answer` is printed. File focus: `src/nl2sql_agent/cli.py`, `src/nl2sql_agent/agent/workflow.py`.
2. State-contract drill: Write the minimal `AgentState` keys required for `write_sql()` to run without missing context. File focus: `src/nl2sql_agent/agent/state.py`, `src/nl2sql_agent/agent/workflow.py`.
3. Security policy scenario: Explain what happens if `allow_joins=False` and the model generates a JOIN query. File focus: `src/nl2sql_agent/security/sql_validator.py`.
4. Retry logic reasoning: For `retry_count=2`, `max_retries=3`, and execution error present, what node runs next and why? File focus: `src/nl2sql_agent/agent/workflow.py`.
5. Data contract check: What are the exact types for `columns`, `raw_rows`, and `row_count` returned by `execute_sql()`? File focus: `src/nl2sql_agent/db/database.py`, `src/nl2sql_agent/agent/workflow.py`.
6. Config mapping task: Map one settings field to an environment variable and describe where it affects runtime behavior. File focus: `src/nl2sql_agent/config/settings.py`.
7. UI behavior task: Explain why live `list_models()` discovery is Ollama-only and how hosted model choices are supplied. File focus: `src/nl2sql_agent/ui/streamlit_app.py`, `src/nl2sql_agent/ui/components.py`.
8. Extension design: Trace the implemented xAI path from settings and credentials through the factory and sidebar. File focus: `src/nl2sql_agent/config/settings.py`, `src/nl2sql_agent/llm/factory.py`, `src/nl2sql_agent/ui/components.py`.
9. Approval flow: Explain why editing SQL in Streamlit cannot bypass the guardian. File focus: `src/nl2sql_agent/ui/streamlit_app.py`, `src/nl2sql_agent/agent/workflow.py`.
10. Evaluation scoring: Explain why reference-result comparison accepts multiple correct SQL forms. File focus: `src/nl2sql_agent/evaluation/runner.py`, `src/nl2sql_agent/evaluation/data/demo.jsonl`.
11. Cost accounting: Explain how per-call usage differs from a run total, and why UI budget alerts do not enforce the benchmark driver's US\$2 ledger ceiling. File focus: `llm/pricing.py`, `llm/budget.py`, `ui/streamlit_app.py`.

5.3 Solution outlines

1. Exercise 1 outline: `main -> parse_args -> cmd_ask -> _build_agent -> get_settings -> build_chat_model -> NL2SQLAgent.run -> get_workflow/invoke -> final dict -> print(final_answer)`.
2. Exercise 2 outline: Minimum practical keys are `question`, `schema`; optional prior context is `error` or `sql_unsafe_reason`; `retry_count` defaults to 0 if missing.
3. Exercise 3 outline: `check_security` calls `prepare_sql`; the policy raises `SQLValidationError("JOIN clauses are not allowed.")`; the node returns `policy_blocked`; routing stops SQL retries and renders the error without execution.
4. Exercise 4 outline: `route_after_execute` checks `err` and `retry < max_retries`; with `2 < 3`, it routes to "writer" for another SQL rewrite attempt.

5. Exercise 5 outline: `execute_sql` returns `columns: list[str], raw_rows: list[tuple[object, ...]], row_count: int`.
6. Exercise 6 outline: Example: `Settings.db_max_rows` from `NL2SQL_DB_MAX_ROWS` influences `Database(max_rows=...)`; `execute()` fetches `max_rows + 1` then truncates.
7. Exercise 7 outline: `list_models()` performs a network call only after `refresh_trigger_ollama` is set. Hosted providers read deterministic choices from the central settings catalog; Hugging Face additionally accepts a custom repository ID.
8. Exercise 8 outline: `Provider` includes `xai`; `env_var_for()` maps `XAI_API_KEY`; `_build_xai()` targets the fixed HTTPS endpoint with medium reasoning; the sidebar exposes only `grok-4.6`.

The same trace for Agnes is `Provider` → `AGNESAI_API_KEY` → `_build_agnes()` → fixed API Hub endpoint with Chat Completions Thinking → `agnes-3.0-flash` as the sidebar default.
9. Exercise 9 outline: `execute_prepared()` replaces the candidate with the edited SQL, calls `check_security()` again, and only reaches `execute_sql()` when the second validation and preflight succeed.
10. Exercise 10 outline: The evaluator executes curated reference SQL read-only and compares result values. SQL aliases, formatting, and equivalent query structure do not have to match; ordered cases still require the requested row order.
11. Exercise 11 outline: `write_sql()` and the legacy model branch of `summarize_result()` retain per-call provider usage in `AgentState.usage_records`; `effective_pricing_rule()` selects the rule, and `calculate_cost()` prices the actual mode for each call, including cache and long-context adjustments. Approved runs freeze the price snapshot. Unknown Ollama or custom Hugging Face models remain unpriced until a matching rule is configured.

Verification checklist

1. Can you explain the full node order in `NL2SQLAgent` and both routing decisions?
2. Can you describe exactly how unsafe SQL is blocked before execution?
3. Can you enumerate the most important `AgentState` keys and what writes each one?
4. Can you explain how retries are triggered and when they stop?
5. Can you show where schema creation and seed insertion occur?
6. Can you explain how provider/model selection flows from UI/CLI into `build_chat_model()`?
7. Can you point to where final answer fallback behavior is implemented?
8. Can you map at least five environment settings to their runtime effects?
9. Can you explain how uploaded databases stay session-scoped and read-only?
10. Can you interpret the accuracy, safety, latency, retry, and token fields in an evaluation report?
11. Can you explain the UI cost formula and why conditional provider discounts are not applied to aggregate run usage?